

# USEFUL INTERFACES, THOUGHTFUL CODE.

복잡한 요구를 읽기 쉬운 화면과 다루기 쉬운 코드로 번역합니다. 사용자에게는 자연스럽고, 동료에게는 설명 가능한 인터페이스를 만드는 프론트엔드 개발자입니다.

01 POINT OF VIEW

## 화면은 예쁘게 끝나지 않고, 오래 다룰 수 있어야 합니다.

프론트엔드 개발은 요구사항을 화면으로 옮기는 일에서 끝나지 않는다고 생각합니다. 오마고치에서는 게임 시스템 기획과 아트부터 37개 화면, React Island, BFF 연동까지 모든 결과가 사용자에게 닿는 통합 지점을 맡았습니다.

새로운 도구를 빠르게 붙이는 것보다 팀이 이유를 설명할 수 있는 선택을 선호합니다. 실제로 모바일 전용 Home을 구현한 뒤에도 문제의 원인이 라우트가 아니라 레이아웃 모델이라고 판단해 폐기했고, 하나의 fluid stage로 다시 설계했습니다.

| 01 / STRUCTURE                                                               | 02 / INTERACTION                                                        | 03 / DELIVERY                                                          |
|------------------------------------------------------------------------------|-------------------------------------------------------------------------|------------------------------------------------------------------------|
| <b>변화에 강한 구조</b><br>반복되는 화면을 복사하지 않고, 변하는 축을 찾아 재사용 가능한 컴포넌트와 데이터 모델로 정리합니다. | <b>예측 가능한 흐름</b><br>로딩, 빈 화면, 오류, 키보드와 터치까지 실제 사용 흐름을 기준으로 인터랙션을 설계합니다. | <b>끝까지 닫는 구현</b><br>브라우저에서 잘 보이는 것을 넘어 성능, 접근성, 반응형, 운영과 유지보수까지 확인합니다. |

# 보이는 화면과 그 뒤의 규칙을 함께 설계합니다.

대표 프로젝트 오마고치에서 내린 프론트엔드 의사결정과 공개 작업을 정리했습니다.

## 01

10-WEEK TEAM PROJECT · 8 PEOPLE

2026.07 - 2026.08

## Omagotchi

게임 시스템 기획 · UI/UX · 아트 · 프론트엔드 전체 · 백엔드 6개 도메인

학습 공간의 출결과 학습 시간을 캐릭터 육성, 레벨, 퀘스트로 연결한 게임화 학습 관리 서비스입니다. 팀의 모든 도메인 결과가 화면으로 수렴하는 통합 지점에서 기획부터 프론트엔드 구현, BFF 연동까지 수직 전 구간을 맡았습니다.

|                 |                          |                  |                  |
|-----------------|--------------------------|------------------|------------------|
| 37              | 22 / 80                  | 141              | 320-1440         |
| THYMELEAF PAGES | STORYBOOK FILES / STATES | CHARACTER ASSETS | RESPONSIVE RANGE |

### 핵심 구현

4가지

- React 19 Island와 Thymeleaf를 연결해 기존 서버 계약을 유지하면서 핵심 Home UI를 점진적으로 전환
- 8개 컬러 토큰과 공통 컴포넌트 7종을 정의하고 80개 UI 상태를 Storybook에서 서버 없이 검증
- HttpOnly 세션을 브라우저 밖 BFF에서 Bearer 토큰으로 변환하고 8개 도메인의 오류 경계를 화면 상태로 통합
- 11개 캐릭터 상태와 8색 팔레트를 직접 설계하고, 정적 몸통과 눈 GIF를 분리해 에셋 제작 비용을 구조적으로 절감

### FRONTEND DECISIONS

선택의 이유를 펼쳐보세요

#### RESPONSIVE ARCHITECTURE

#### 두 벌의 Home 대신 하나의 fluid stage

모바일 전용 페이지까지 구현했지만 태블릿과 가로 모드가 계속 어긋났습니다. 증상이 아니라 레이아웃 모델을 원인으로 보고, /home 단일 진입점과 React Island로 전환해 기능 중복과 viewport redirect를 없앴습니다.

#### COMPONENT BOUNDARY

#### 공통 UI와 Home 전용 UI를 2계층으로 분리

순수 표현 컴포넌트와 data-\* DOM 계약에 종속된 Home 컴포넌트를 분리해, 공통 라이브러리가 특정 화면의 로직에 오염되지 않도록 했습니다.

#### STATE VERIFICATION

#### 오류와 빈 상태를 서버보다 먼저 확정

실제 서버 조건을 만들기 어려운 loading, empty, error 상태를 Storybook 80개 사례로 고정하고 실제 화면에 연결했습니다.

#### DESIGN SYSTEM

#### 프레임워크보다 필요한 기능만 선택

기존 전역 CSS와 픽셀 아트 톤을 지키기 위해 CSS 프레임워크 도입을 폐기하고, 접근성이 필요한 Radix UI와 상태 전환용 Motion만 제한적으로 채택했습니다.

## 01 Thymeleaf 안에 React Island를 어떻게 연결했나

서버가 소유한 라우트와 세션 계약은 유지하고, 상호작용이 많은 Home 영역만 독립적으로 마운트했습니다. 서버 템플릿은 초기 경계와 data-\* 계약을 제공하고 React는 그 안의 상태와 렌더링을 책임집니다.

```
const root = document.querySelector('[data-home-island]');

if (root) {
  createRoot(root).render(
    <HomeIsland userId={root.dataset.userId} />
  );
}
```

실제 구현을 설명하기 위해 축약한 구조 예시입니다.

## 02 로딩·빈 화면·오류를 하나의 상태 모델로 다루기

API 응답을 곧바로 JSX 조건문에 흘려버리지 않고 화면이 가질 수 있는 상태를 먼저 한정했습니다. Storybook에서도 같은 상태 모델을 사용해 서버 없이 경계 상황을 재현했습니다.

```
type ViewState<T> =
  | { status: 'loading' }
  | { status: 'empty' }
  | { status: 'error'; message: string }
  | { status: 'ready'; data: T };
```

상태 분기를 설명하기 위한 대표 타입입니다.

## 03 인증 변환은 브라우저가 아니라 BFF의 책임으로

브라우저에는 HttpOnly 세션만 남기고, 내부 API가 요구하는 Bearer 토큰 변환은 서버 경계에서 처리했습니다. UI 컴포넌트가 인증 세부사항을 알지 않게 해 화면 로직과 보안 책임을 분리했습니다.

```
const session = await readHttpOnlySession(request);
const response = await api.fetch('/home', {
  headers: { Authorization: `Bearer ${session.token}` }
});

return mapToViewState(response);
```

토큰 값과 실제 서버 구현을 노출하지 않은 흐름 예시입니다.

## Data-driven Portfolio

콘텐츠와 표현 레이어를 분리한 포트폴리오. 하나의 데이터 파일만 수정하면 섹션과 내비게이션, 공유 메타데이터가 함께 이어지는 구조입니다.

### 핵심 구현

3가지

- 의미 있는 HTML 구조와 키보드 탐색, 모션 감소 설정을 기본값으로 설계
- React/Vinext 기반 Cloudflare Worker 호환 배포 구조
- 반응형 레이아웃, 다크 모드, 인쇄 스타일, Open Graph 공유 카드 지원

## Title.isEmpty Systems

Unity에서 2D Blink 이동을 구현한 인터랙션 시스템입니다. 순간 이동의 손맛을 만들기 위해 Hit-stop, l-frame, Normal-offset을 하나의 동작 흐름으로 구성했습니다.

### 핵심 구현

3가지

- 입력에 대한 즉각적인 피드백과 충돌 경계의 안정성을 함께 고려
- 화면 전환이 아닌 게임 인터랙션에서도 상태와 타이밍을 설계
- 사용자가 체감하는 동작을 작은 시스템 단위로 분리

기술 이름보다, 어디에 왜  
쓰는지를 중요하게 봅니다.

### 화면의 구조와 동작

## Frontend Core

- + React 19
- + JavaScript
- + Thymeleaf
- + Semantic HTML
- + Modern CSS

### 일관된 사용자 경험

## Interface Design

- + Storybook
- + Radix UI
- + Motion
- + Figma
- + Responsive UI

완성도를 만드는 기준

## Product Quality

- + Accessibility
- + 320~1440px QA
- + Loading / Empty / Error
- + Design Tokens

경계를 이해하는 도구

## Beyond the UI

- + Java 21
- + PostgreSQL
- + Spring Boot
- + Docker
- + BFF

### 04 HOW I WORK

## 작게 만들고, 빨리 확인하고, 이유를 남깁니다.

01

### DEFINE

문제를 먼저 같은 언어로 맞춥니다.

요구사항에서 사용자 목표, 예외 상황, 성공 기준을 분리해 구현 전에 불확실성을 줄입니다.

02

### MODEL

상태와 경계를 설계합니다.

컴포넌트보다 데이터 흐름을 먼저 보고, 무엇을 어디에서 책임질지 결정합니다.

03

### VERIFY

브라우저에서 실제 흐름을 확인합니다.

정상 화면만 보지 않고 로딩, 빈 상태, 오류, 작은 화면과 키보드 사용까지 확인합니다.

04

### DOCUMENT

다음 사람이 이해할 단서를 남깁니다.

선택의 이유와 트레이드오프를 코드와 문서에 남겨 팀의 다음 판단을 가볍게 만듭니다.

05

LET'S  
BUILD  
SOMETHING  
USEFUL

## 좋은 화면은 좋은 대화에서 시작됩니다.

제품과 사용자의 사이를 더 단순하고 분명하게 만드는 일이라면 즐겁게 이야기하겠습니다.

[mjm3204@naver.com](mailto:mjm3204@naver.com)